

Week 3 – Async JavaScript: Detailed Exercises (Days 15-21)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 31 Aug 2026 – 6 Sep 2026 (Days 15-21)

 **Your slots (IST):** Mon-Fri **7:30-8:30pm & 9:00-10:00pm** · Sat-Sun **9:00am-12:00pm, 3:00-5:30pm, 6:00-8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import `calendar/mern-mastery-study.ics`

Roadmap: [ROADMAP.md](#) · Theory: [01-javascript-complete.md §7-10](#) · Index: [week-03-exercises.md](#)

Day 15 – Event Loop Fundamentals

Exercise 15.1: Event Loop Prediction (Core)

Problem statement

Predict output order, then run in Node:

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
Promise.resolve().then(() => {
  console.log("D");
  setTimeout(() => console.log("E"), 0);
});
console.log("F");
```

Learning objective

Order: sync → microtasks (Promises) → macrotasks (setTimeout).

Prerequisites

- Theory: [01-javascript-complete.md §7 Asynchronous JavaScript](#)

Step-by-step approach

1. Write order on paper before running.
2. Mark sync: A, F.
3. Drain microtask queue: C, D (nested timeout schedules E).
4. Macrotask: B, then E.
5. Run `node event-loop.js` and compare.
6. Repeat with `async/await` replacing `Promise.then`.
7. Add 5 more variants in `event-loop-drills.js`.

Hints

1. Microtasks run before next macrotask.
2. `setTimeout(0)` is not instant — queued after current stack + microtasks.
3. Node vs browser mostly same for this snippet.

Exact Answer Output order: **A → F → C → D → B → E**

- Sync: A, F run first
- Microtasks: C, D (Promise.then queue drained before macrotasks)
- Macrotasks: B (setTimeout from line 2), then E (setTimeout scheduled inside D's microtask)

Complete Solution

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
Promise.resolve().then(() => {
  console.log("D");
  setTimeout(() => console.log("E"), 0);
});
console.log("F");
// Output: A, F, C, D, B, E
```

Expected output / acceptance criteria

- Correct order: A, F, C, D, B, E
- 10 prediction problems with ≥ 8 correct before running.

Where to practice

- Loupe: <http://latentflip.com/loupe/>
- MDN Event Loop: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Event_loop

- Local: `mern-mastery/week-03/day-15/`

Common mistakes

- Thinking `setTimeout(0)` runs before Promises.
- Forgetting nested timeout in D's callback.

80 LPA Interview Twist

Google L4 asks 10-snippet marathons mixing `queueMicrotask`, `setImmediate`, and `process.nextTick` in Node — you must draw the microtask vs macrotask queue on a whiteboard under time pressure.

Interview connection

"What prints first?" is a standard JS screening question; explains async bug timing.

Time estimate: 60 minutes

Day 16-17 – Promises & async/await

Exercise 16.1: Promise Chain Drills

Problem statement

Write three promise chains:

1. Fetch mock data with delay, double value, catch errors
2. Sequential async steps (login → fetch profile → log)
3. Convert chain to async/await with try/catch

Learning objective

Compose async work; handle rejection propagation.

Prerequisites

- Theory: [01-javascript-complete.md §7 Promises & async/await](#)

Step-by-step approach

1. Create `delay(ms)` returning Promise.
2. Chain `.then` ensuring return values pass down.
3. Add `.catch` at end; throw in middle to test.
4. Rewrite with async/await — same behavior.
5. Compare error stacks.

6. Never use `.then` on async function without need.

Hints

1. `return` inside then passes to next then.
2. async functions always return Promises.
3. try/catch in async replaces `.catch` for awaited code.

Exact Answer

- Chain 1: delay → double → log (e.g., `2 → 4`)
- Chain 2: login → profile → log (sequential order preserved)
- Chain 3: async/await produces identical output to chains 1–2
- Errors propagate to single catch handler

Complete Solution

```
const delay = (ms, value) =>
  new Promise((resolve) => setTimeout(() => resolve(value), ms));

// Chain 1: delay → double → catch
delay(100, 2)
  .then((val) => val * 2)
  .then((val) => console.log("Chain1:", val))
  .catch((err) => console.error(err));

// Chain 2: sequential login → profile
const login = () => delay(50, { userId: 1 });
const fetchProfile = (id) => delay(50, { id, name: "Alex" });

login()
  .then((session) => fetchProfile(session.userId))
  .then((profile) => console.log("Chain2:", profile))
  .catch(console.error);

// Chain 3: async/await equivalent
async function chain3() {
  try {
    const session = await login();
    const profile = await fetchProfile(session.userId);
    console.log("Chain3:", profile);
  } catch (err) {
    console.error(err);
  }
}
chain3();
```

Expected output / acceptance criteria

- All three chains log expected sequence.
- Errors caught and logged once.

Where to practice

- Local: `week-03/day-16/promise-chains.js`
- MDN Promise: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

Common mistakes

- Forgetting to return promise in then (broken chain).
- Catching too early and swallowing errors.

80 LPA Interview Twist

Meta asks you to implement Promise retry with exponential backoff, or convert a callback-based API to Promise while preserving error stacks — production-grade async, not toy chains.

Interview connection

"Convert callback to Promise" and fetch error handling in React apps.

Time estimate: 75 minutes

Day 18 – Promise.all Polyfill

Exercise 18.1: promiseAll Implementation

Problem statement

Implement `promiseAll(promises)` — resolves to array of results in order; rejects if any input rejects. Handle empty array.

Learning objective

Coordinate parallel async; preserve order vs completion order.

Prerequisites

- Theory: [01-javascript-complete.md §7](#)

Step-by-step approach

1. Return new Promise wrapper.
2. If `promises.length === 0`, resolve `[]`.
3. Track `remaining` count and `results` array.

4. `promises.forEach((p, i) => Promise.resolve(p).then(val => { results[i]=val; if(--remaining===0) resolve(results); })).`
5. On any reject, reject immediately (optional: Promise.all behavior).
6. Test: all resolve, one reject, non-promise values, empty array.
7. Compare with `Promise.allSettled` and `Promise.race`.

Hints

1. Wrap with `Promise.resolve(p)` for non-promises.
2. Store by index `i` to preserve order.
3. Empty array resolves synchronously to `[]`.

Exact Answer

- `promiseAll([Promise.resolve(1), 2, Promise.resolve(3)])` → `[1, 2, 3]`
- `promiseAll([])` → `[]`
- One rejection → entire promise rejects with that reason
- Time: **O(n)**, preserves input order regardless of completion order

Complete Solution

```
function promiseAll(promises) {
  return new Promise((resolve, reject) => {
    if (promises.length === 0) return resolve([]);

    const results = new Array(promises.length);
    let remaining = promises.length;

    promises.forEach((p, i) => {
      Promise.resolve(p).then(
        (val) => {
          results[i] = val;
          if (--remaining === 0) resolve(results);
        },
        reject
      );
    });
  });
}

promiseAll([Promise.resolve(1), 2, Promise.resolve(3)]).then(console.log); // [1,
promiseAll([]).then(console.log); // []
```

Line-by-line explanation (javascript)

Line	What it does
<code>promiseAll([Promise.resolve(1), 2, Promise.resolve(3)]).then(console.log);</code>	<code>[1, 2, 3]</code>
<code>promiseAll([]).then(console.log);</code>	<code>[]</code>

Expected output / acceptance criteria

```
promiseAll([Promise.resolve(1), 2, Promise.resolve(3)]); // [1,2,3]
promiseAll([]); // []
// One rejection → entire promiseAll rejects
```

Where to practice

- Local: `week-03/day-18/promiseAll.js`

Common mistakes

- Pushing results in completion order (wrong order).
- Not handling empty array (hang forever).

80 LPA Interview Twist

Follow-ups: implement `Promise.allSettled`, `Promise.race`, and concurrency-limited pool (max 5 parallel) — Google uses these to test production fetch orchestration patterns.

Interview connection

Parallel API fetches in Express/Next — batch requests with error policy.

Time estimate: 50 minutes

Day 20 – Debounce & Throttle

Exercise 20.1: Debounce

Problem statement

Implement `debounce(fn, delay)` — invoke `fn` only after `delay` ms of no calls.

Learning objective

Timer cancellation via closure; leading vs trailing edge optional.

Prerequisites

- Theory: [01-javascript-complete.md §10 Interview Implementations](#)

Step-by-step approach

1. Return function that clears previous `timeoutId`.
2. Set new `setTimeout(() => fn.apply(this, args), delay)`.
3. Simulate search input: rapid calls → one API mock.
4. Optional: `immediate` flag for leading edge.
5. Write Jest-style manual tests with fake timers if available.
6. Compare with lodash debounce options.

Hints

1. Store `timeoutId` in closure.
2. Use `...args` and `apply` to forward arguments.
3. `clearTimeout` before each reschedule.

Exact Answer

- 10 rapid calls within 100ms with 300ms delay → **exactly 1 execution** after the last call (+ 300ms)
- Trailing-edge debounce: fn runs once quiet period ends

Complete Solution

```
function debounce(fn, delay) {
  let timeoutId;
  return function (...args) {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => fn.apply(this, args), delay);
  };
}

// Demo
let count = 0;
const debouncedLog = debounce((msg) => {
  count++;
  console.log(`Executed ${count}x:`, msg);
}, 300);

for (let i = 0; i < 10; i++) debouncedLog(`call-${i}`);
// Only one log ~300ms after last call
```

Expected output / acceptance criteria

- 10 rapid calls in 100ms with 300ms delay → 1 execution after last call.

Where to practice

- Local: `week-03/day-20/debounce.js`
- CSS-Tricks debounce: <https://www.joshwcomeau.com/snippets/javascript/debounce/>

Common mistakes

- Losing `this` context without `apply/bind`.
- Not clearing timer (multiple executions).

80 LPA Interview Twist

Google asks for leading + trailing debounce, cancel method, and `maxWait` (lodash-style) — search autocomplete at scale requires all three options explained with tradeoffs.

Interview connection

Search autocomplete, resize handlers — explain debounce vs throttle tradeoffs.

Time estimate: 45 minutes

Exercise 20.2: Throttle

Problem statement

Implement `throttle(fn, limit)` — at most one call per `limit` ms.

Learning objective

Time-based gating vs debounce's quiet-period triggering.

Step-by-step approach

1. Track `lastRun` timestamp or `inThrottle` flag.
2. If elapsed $\geq$ limit, run immediately and update `lastRun`.
3. Else schedule trailing call optional.
4. Test scroll simulation: 100 events/sec $\rightarrow$ $\sim$ 10/sec with 100ms limit.
5. Document when throttle beats debounce (scroll, mousemove).

Hints

1. Simple version: ignore calls inside window.
2. Leading + trailing variants exist — pick one and document.
3. Use `Date.now()` for elapsed time.

Exact Answer

- Under 100 events/sec with 100ms limit $\rightarrow$ at most **$\sim$ 10 executions/sec**

- First call in each window executes immediately (leading-edge); calls inside window are ignored or queued

Complete Solution

```
function throttle(fn, limit) {
  let lastRun = 0;
  return function (...args) {
    const now = Date.now();
    if (now - lastRun >= limit) {
      lastRun = now;
      fn.apply(this, args);
    }
  };
}

// Demo: simulate scroll events
let execCount = 0;
const throttled = throttle(() => execCount++, 100);

const interval = setInterval(throttled, 10); // fire every 10ms
setTimeout(() => {
  clearInterval(interval);
  console.log(`Executions in 1s: ${execCount}`); // ~10
}, 1000);
```

Line-by-line explanation (javascript)

Line	What it does
<code>const interval = setInterval(throttled, 10);</code>	fire every 10ms
<code>console.log(Executions in 1s: \${execCount});</code>	~10

Expected output / acceptance criteria

- Throttled fn runs at most once per `limit` ms under load.

Where to practice

- Local: `week-03/day-20/throttle.js`

Common mistakes

- Implementing debounce instead of throttle.
- Drift accumulation with `setInterval` approach.

80 LPA Interview Twist

Meta connects throttle to rate limiter LLD — expect "implement throttle with trailing call guarantee" and compare token bucket vs fixed window for API gateway design.

Interview connection

Rate limiting UI events and API calls — connects to backend rate limiter LLD.

Time estimate: 45 minutes

Day 20 – Implement bind

Exercise 20.3: Function.prototype.myBind

Problem statement

Implement `myBind(context, ...args)` without native `.bind`. Support partial application and `new` operator.

Learning objective

Hard bind pattern; distinguish bound call vs constructor call.

Prerequisites

- Theory: [01-javascript-complete.md §3 this](#)

Step-by-step approach

1. Save `originalFn = this`.
2. Return function `bound(...laterArgs)`.
3. If `new.target`, call `originalFn` with merged args (constructor path).
4. Else `originalFn.apply(context, [...args, ...laterArgs])`.
5. Optional: copy prototype with `Object.create` for `instanceof`.
6. Test partial args and `new boundFn()`.

Hints

1. `bind` returns a new function always.
2. `new` ignores bound `this` — use `new.target` check.
3. Partial application prepends bound args.

Exact Answer

- Normal call: bound function uses provided `context` as `this`

- Partial args prepended: `myBind(fn, ctx, 1)(2) == fn.call(ctx, 1, 2)`
- With `new`: bound `this` ignored; original constructor runs

Complete Solution

```
Function.prototype.myBind = function (context, ...boundArgs) {
  const originalFn = this;

  function bound(...laterArgs) {
    const isNew = this instanceof bound;
    return originalFn.apply(
      isNew ? this : context,
      [...boundArgs, ...laterArgs]
    );
  }

  bound.prototype = Object.create(originalFn.prototype);
  return bound;
};

function greet(greeting, name) {
  return `${greeting}, ${this.title} ${name}`;
}

const obj = { title: "Mr." };
const boundGreet = greet.myBind(obj, "Hello");
console.log(boundGreet("Smith")); // "Hello, Mr. Smith"

function Person(name) { this.name = name; }
const BoundPerson = Person.myBind(null, "Test");
const p = new BoundPerson();
console.log(p.name); // "Test"
```

Line-by-line explanation (javascript)

Line	What it does
<code>console.log(boundGreet("Smith"));</code>	"Hello, Mr. Smith"
<code>console.log(p.name);</code>	"Test"

Expected output / acceptance criteria

- Bound function has correct `this` in normal call.
- Works with `new` when required by spec exercise variant.

Where to practice

- Local: `week-03/day-20/myBind.js`

Common mistakes

- Only handling apply case, breaking `new`.
- Arrow functions can't be bound (have no own `this`).

80 LPA Interview Twist

Google asks implement `call`, `apply`, and `bind` together with spec-correct `new` behavior — often as a 30-minute live coding round without lodash reference.

Interview connection

Classic JS interview alongside call/apply; underpins React legacy class binding.

Time estimate: 55 minutes

Days 20-21 – CLI Todo Project (8-Step Build)

Project: CLI Todo App with File Persistence

Problem statement

Build a Node.js CLI todo application:

- Commands: `add`, `list`, `complete`, `delete`
- Persist to `todos.json`
- Async file read/write

Learning objective

Apply async I/O, CLI parsing, JSON persistence, error handling.

Prerequisites

- Theory: [01-javascript-complete.md §7-10](#)
- Node fs/promises basics

Step-by-step approach (8 steps)

Step 1 — Project scaffold (30 min)

Create `cli-todo/` with `package.json`, `"type": "module"` or CommonJS consistently. Entry: `index.js`. Install none or `minimist` for args. Create empty `todos.json` as `[]`.

Step 2 — File I/O layer (45 min)

Implement `readTodos()` and `writeTodos(todos)` using `fs.promises`. Handle missing file → return `[]`. Wrap JSON parse errors with clear message.

Step 3 — Todo model (30 min)

Shape: `{ id, title, completed, createdAt }`. `generateId()` via `crypto.randomUUID()` or incremental max id.

Step 4 — Command: add (30 min)

Parse `node index.js add "Buy milk"`. Append todo, save file, print confirmation.

Step 5 — Command: list (30 min)

Print table: id, status checkbox, title. Filter flags optional: `--done`, `--pending`.

Step 6 — Command: complete & delete (45 min)

`complete <id>` toggles or sets completed. `delete <id>` removes. Validate id exists; exit code 1 on error.

Step 7 — CLI router & help (30 min)

Switch on `process.argv[2]`. Print usage for unknown commands. `node index.js help`.

Step 8 — Polish & tests (60 min)

Add input validation, concurrent write safety (simple mutex or queue), README with examples. Manual test script covering full CRUD.

Hints

1. Start with sync fs only if stuck — refactor to async immediately after.
2. Always read-modify-write entire file for simplicity at this stage.
3. Use `process.argv.slice(2)` for command args.

Exact Answer

```
node index.js add "Learn event loop"  # → Added todo id=<uuid>
node index.js list                    # → table of todos
node index.js complete <id>          # → marked complete
node index.js delete <id>            # → removed
# todos.json persists across restarts
```

Complete Solution

```
// index.js - CLI Todo with file persistence
import { readFile, writeFile } from "fs/promises";
import { randomUUID } from "crypto";

const FILE = "todos.json";

async function readTodos() {
  try {
    const data = await readFile(FILE, "utf-8");
    return JSON.parse(data);
  } catch {
    return [];
  }
}

async function writeTodos(todos) {
  await writeFile(FILE, JSON.stringify(todos, null, 2));
}

function generateId() {
  return randomUUID();
}

async function add(title) {
  const todos = await readTodos();
  const todo = { id: generateId(), title, completed: false, createdAt: new Date() };
  todos.push(todo);
  await writeTodos(todos);
  console.log(`Added: [${todo.id}] ${title}`);
}

async function list() {
  const todos = await readTodos();
  if (todos.length === 0) return console.log("No todos.");
  todos.forEach((t) => console.log(`${t.completed ? "[x]" : "[ ]"} ${t.id} - ${t.title}`));
}

async function complete(id) {
  const todos = await readTodos();
  const todo = todos.find((t) => t.id === id);
  if (!todo) { console.error("Not found"); process.exit(1); }
  todo.completed = true;
  await writeTodos(todos);
  console.log(`Completed: ${todo.title}`);
}

async function remove(id) {
  const todos = await readTodos();
  const idx = todos.findIndex((t) => t.id === id);
  if (idx === -1) { console.error("Not found"); process.exit(1); }
  todos.splice(idx, 1);
}
```

```
    await writeTodos(todos);
    console.log(`Deleted: ${id}`);
  }

  const [cmd, ...args] = process.argv.slice(2);
  switch (cmd) {
    case "add": await add(args.join(" ")); break;
    case "list": await list(); break;
    case "complete": await complete(args[0]); break;
    case "delete": await remove(args[0]); break;
    default:
      console.log("Usage: node index.js <add|list|complete|delete> [args]");
  }
}
```

Expected output / acceptance criteria

```
node index.js add "Learn event loop"
node index.js list
# → shows pending todo with id
node index.js complete <id>
node index.js delete <id>
# todos.json reflects all changes after restart
```

Where to practice

- Node fs docs: <https://nodejs.org/api/fs.html#promises-api>
- Local: `mern-mastery/week-03/cli-todo/`

Common mistakes

- Forgetting to await write before exit (truncated file).
- Not handling empty argv (show help).
- Duplicate ids on crash mid-write.

80 LPA Interview Twist

Amazon/Google extend this to atomic writes (write temp + rename), file locking for concurrent CLI processes, and SQLite migration path — testing production Node I/O patterns beyond toy JSON files.

Interview connection

Demonstrates Node fundamentals before Express week; file I/O parallels DB persistence patterns.

Time estimate: 6–8 hours (Days 20–21 weekend)

Day 21 – Event Loop Marathon & Review

Exercise 21.1: Ten Event Loop Problems

Problem statement

Complete 10 printed-order puzzles mixing `setTimeout`, `setImmediate` (Node), Promises, `async/await`, and `queueMicrotask`.

Learning objective

Fluency under interview pressure.

Step-by-step approach

1. Source 10 snippets from notes + Loupe.
2. Predict all before any execution.
3. Score $\geq 8/10$; redo missed ones next day.
4. Write one-sentence rule per mistake.

Exact Answer

- Target score: $\geq 8/10$ correct predictions before running code
- Core rule: Sync $\rightarrow$ Microtasks (Promises, `queueMicrotask`) $\rightarrow$ Macrotasks (`setTimeout`, `setImmediate`)
- Node addition: `process.nextTick` runs before Promise microtasks

Complete Solution

```
// Sample drill answers (document all 10 in event-loop-answers.md):

// Drill 1 – Core (Exercise 15.1)
// A, F, C, D, B, E

// Drill 2 – async/await
async function drill2() {
  console.log("1");
  await Promise.resolve();
  console.log("2");
}
console.log("0");
drill2();
console.log("3");
// 0, 1, 3, 2

// Drill 3 – queueMicrotask vs Promise
console.log("A");
queueMicrotask(() => console.log("B"));
Promise.resolve().then(() => console.log("C"));
console.log("D");
// A, D, B, C (B before C – queueMicrotask runs first in Node)
```

Expected output / acceptance criteria

- Document `week-03/event-loop-answers.md` with orders + explanations.

80 LPA Interview Twist

At Meta E4+, event loop questions include "design an async task scheduler with priority queues" — the prediction drills are warmup for building internal concurrency primitives.

Where to practice

- <http://latentflip.com/loupe/>
- Node docs setImmediate: <https://nodejs.org/api/timers.html#setimmediatecallback-args>

Time estimate: 2 hours

Week 3 Completion Checklist

Item	Done
Event loop core + 10 drills	☐
debounce + throttle	☐
promiseAll polyfill	☐
myBind	☐
CLI Todo 8 steps complete	☐
Code in week-03/	☐

Total estimated time: ~22–26 hours